

Factorial Time complexity using Substitution Method

Now, there are two ways to do :

1st Way

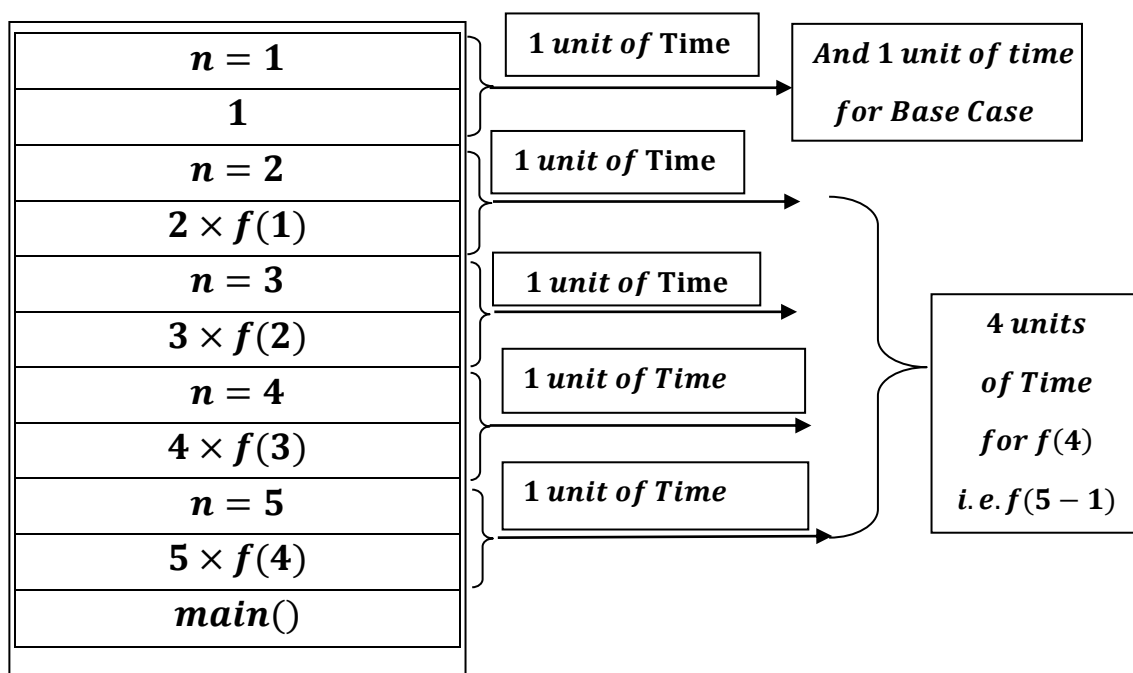
Base Cases:

$T(0) = 1$, when $n = 0$ i. e. when $n = 0$, it returns 1 and it takes 1 unit of time .

$T(1) = 1$, when $n = 1$ i. e. when $n = 1$, it returns 1 and it takes 1 unit of time .

and now for $n > 0$.

$F(n - 1) \times n \Rightarrow$ Just take the last push for $F(5)$:



$F(n - 1) \times n$ runs each at constant i. e. constant unit of time takes $T(n - 1)$ times and as it reaches $T(1)$ it executes an extra constant time i. e. 1 unit of time .

*Hence total complexity is: $T(n) = T(n - 1) + T(1)$ or,
 $T(n) = T(n - 1) + 1$.*

As each time it multiples it takes a constant amount of time complexity.

$$T(n) = \begin{cases} 1 & , \text{for } (n = 0) \\ 1 & , \text{for } (n = 1) \\ T(n - 1) + T(1) & , \text{for } (n > 1) \end{cases}$$

2nd way :

According to some authors:

Here say base case is 0 only,

```
int factorial(int n)
{
    if (n == 0)
    {
        return 1;
    }

    else
    {
        return n * factorial(n - 1);
    }
}
```

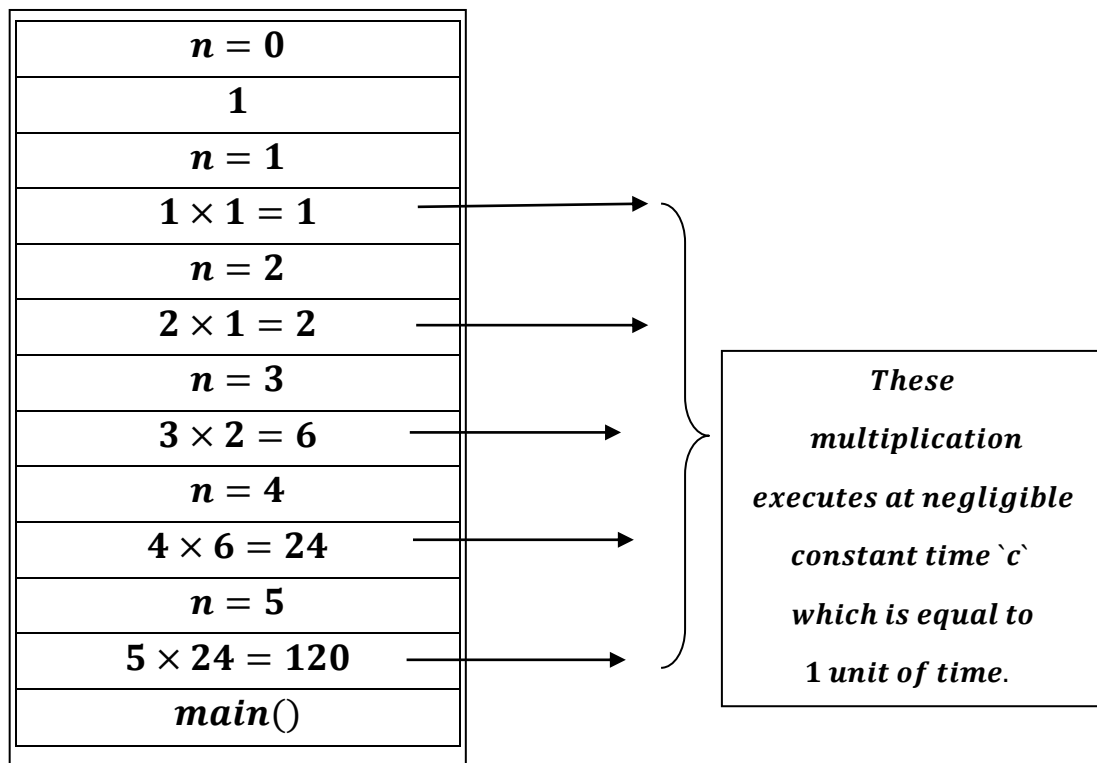
Factorial of (0) = 1 and Factorial of (1) = $1 \times 1 = 1$.

Then with that approach:

$n = 0$	
1	
$n = 1$	
$1 \times f(0)$	
$n = 2$	
$2 \times f(1)$	
$n = 3$	
$3 \times f(2)$	
$n = 4$	
$4 \times f(3)$	
$n = 5$	
$5 \times f(4)$	
$main()$	

Hence $F(n - 1)$ takes $T(n - 1)$ times i.e. upto $n > 0$ or, upto $F(1)$ i.e. $F(4) \rightarrow F(3) \rightarrow F(2) \rightarrow F(1)$.

And,



$$T(n) = T(n - 1) + 1 \text{ or } C, \text{ for } n > 0$$

*To Compute
 $F(n - 1)$*

*To multiply
 $F(n - 1)$ by n*

Which gives the following general recurrence equation: –

$$T(n) = \begin{cases} 1 & , \text{for } (n = 0) \\ T(n - 1) + 1 & , \text{for } (n > 0) \end{cases}$$

But we will move with the 2nd way, as it is accurate i. e.:

$$T(n) = \begin{cases} 1 & , \text{for } (n = 0) \\ T(n - 1) + C & , \text{for } (n > 0) \end{cases}$$

$$T(n) = T(n - 1) + \begin{array}{|c|} \hline \text{To Compute} \\ F(n - 1) \\ \hline \end{array} \begin{array}{|c|} \hline \text{To multiply} \\ F(n - 1) \text{ by } n \\ \hline \end{array} \quad , \text{for } n > 0$$

$$T(n) = T(n - 1) + 1$$

$$\text{Therefore , } T(n - 1) = T(n - 2) + 1$$

Substituting this in $T(n)$ we get:

$$T(n) = (T(n - 2) + 1) + 1 = T(n - 2) + 2$$

Now , $T(n - 2) = T(n - 3) + 1$

Substituting this in $T(n)$ we get:

$$T(n) = T(n - 3) + 1 + 2 = T(n - 3) + 3$$

Now if it runs upto level `k` times we get :

Now , $T(n) = T(n - k) + k$

As, Base Case = $T(0)$, hence , $n - k = 0 \Rightarrow n = k$.

Substituting $k = n$ in $T(n)$ we get:

$$T(n) = T(n - n) + n$$

$$\Rightarrow T(n) = T(0) + n$$

As we know , $T(0) = 1$, substituting this in $T(n)$ we get:

$$\Rightarrow T(n) = 1 + n$$

Applying Big O in the equation:

$$\Rightarrow T(n) = O(1 + n)$$

$$= O(n).$$
